

LAPORAN PRAKTIKUM 4

Analisis Kompleksitas Algoritma



Oleh:

Nama : Nuraida Safitri

NIM : 2025573010038

Jurusan : Teknologi Informasi dan Komputer

Kelas : TI 1D

Lembar Pengesahan

Judul Praktikum	: Perintah dasar linux
Nama	: Nuraida safitri
NIM	:2025583020055
Jurusan	: Teknologi Informasi dan Komputer
Program Studi	: Teknik informatika
Tanggal Percobaan	: 26/04/2026
Tanggal Penyerahan	: 27/04/2026
Dosen Pembimbing	: Muhammad Reza Zulman, M.Sc

Mengetahui,
praktikum

Nama dosen

Nuraida Safitri
Nim. 2025573010038

Muhammad Reza Zulman, M.Sc

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat-Nya laporan tentang menggunakan java script dan node.js ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum serta sebagai bentuk pemahaman terhadap Analisis Kompleksitas Algoritma Big O Notation, Time & Space Complexity saya menyadari laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan. Semoga laporan ini bermanfaat bagi penulis dan pembaca.

Buket Rata, 27 april 2026

Penulis,

NURaida SAFITRI
NIM. 2025573010038

BAB 1

PENDAHULUAN

1.1 Latar belakang

Dalam pemrograman , konsep **Object Oriented Programming (OOP)** menjadi salah satu pendekatan yang banyak digunakan untuk mengelola program yang kompleks. OOP memungkinkan kita untuk mengorganisasi kode menjadi struktur yang lebih teratur, mudah dipahami, dan mudah dikembangkan. .

Pada bahasa pemrograman **JavaScript**, konsep OOP dapat diterapkan menggunakan fitur **class**, constructor, method, inheritance, dan polymorphism. Dengan menggunakan class, programmer dapat membuat blueprint atau cetak biru dari suatu objek yang memiliki properti dan perilaku tertentu.

Pada modul praktikum ini, kita bisa mempelajari cara membuat dan menggunakan **class dan object**, memahami konsep **inheritance**, serta menerapkan **method override dan polymorphism**. Selain itu, konsep tersebut juga digunakan dalam implementasi struktur data seperti **Stack dan Queue**, yang merupakan dasar penting dalam struktur data dan algoritma.

1.2 Tujuan Pembelajaran

Berdasarkan modul, tujuan pembelajaran adalah agar mahasiswa mampu:

- Memahami pentingnya efisiensi algoritma
- Menjelaskan konsep **Time Complexity** dan **Space Complexity**

- Mengetahui dan menuliskan notasi Big O seperti: $O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$, dll
- Menganalisis kompleksitas pada loop tunggal dan nested loop
- Membandingkan performa algoritma secara empiris
- Mengenali kode yang tidak efisien dan melakukan perbaikan (refactor)

BAB II DASAR TEORI

1. Big o notation

Big O digunakan untuk mengukur laju pertumbuhan (growth rate) waktu atau memori algoritma terhadap ukuran input, bukan waktu absolut.

- $O(1)$ → konstan (tidak tergantung input)
- $O(\log n)$ → logaritmik (contoh: binary search)
- $O(n)$ → linear (loop biasa)
- $O(n \log n)$ → linearitmik (sorting optimal)
- $O(n^2)$ → kuadratik (nested loop)
- $O(2^n)$ → eksponensial (sangat lambat)

2. Time complexity

Mengukur waktu eksekusi algoritma.

Contoh:

- Loop tunggal → $O(n)$
- Nested loop → $O(n^2)$
- Rumus langsung → $O(1)$

Dari modul, algoritma dengan hasil sama bisa punya performa berbeda, misalnya:

- Penjumlahan dengan loop → $O(n)$
- Penjumlahan dengan rumus → $O(1)$

3. Time space trade off

Sering terjadi kompromi antara waktu dan memori:

- Algoritma bisa dibuat lebih cepat dengan menggunakan lebih banyak memori
- Atau lebih hemat memori tapi lebih lambat

4. Space complexity

Mengukur penggunaan memori tambahan.

Contoh:

- Variabel biasa $\rightarrow O(1)$
- Membuat array baru $\rightarrow O(n)$
- Rekursi $\rightarrow O(n)$ (karena call stack)

Trade-off yang umum: Banyak algoritma bisa dibuat lebih cepat dengan mengorbankan lebih banyak memori, atau sebaliknya. Ini disebut time-space trade-off. Contoh: memoization membuat Fibonacci dari $O(2n)$ time $\rightarrow O(n)$ time, tapi memerlukan $O(n)$ space tambahan.

5. Pola kompleksitas

Beberapa pola umum:

- Akses array langsung $\rightarrow O(1)$
- Loop $\rightarrow O(n)$
- Nested loop $\rightarrow O(n^2)$
- Rekursi bercabang $\rightarrow O(2^n)$

BAB III

LANGKAH KERJA

3.1 Langkah – Langkah

1. Buka terminal, navigasikan ke root folder repository kamu.
2. Mkdir praktikum -5/ cd praktikum-5
npm init-y
touch semua folder

```

MINGW64:/d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma/praktikum-5
HP@LAPTOP-5IBPM001 MINGW64 ~
$ cd /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ mkdir praktikum-5
mkdir: cannot create directory 'praktikum-5': File exists
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ cd praktikum-5
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
/praktikum-5 (main)
$ npm init -y
Wrote to D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\praktikum-5\packa
ge.json:
{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
/praktikum-5 (main)
$ touch 02 destructuring spread.js
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
/praktikum-5 (main)
$ touch 03-space-complexity
HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
/praktikum-5 (main)
$ touch 04-refactor.js

```

3. Dalam folder praktikum-5/. Buat file baru bernama
01-intro-kompleksitas

kalo foldernya udah ada simpan file lalu jalankan di terminal dengan cara ketik
node 01-intro-kompleksitas

```

PS D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\praktikum-5> node 01-intro-kompleksitas.js
===Perbandingan Waktu Eksekusi===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 0 ms
O(n2) cariPasangan : 0 ms

Hasil sama? true
PS D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\praktikum-5>

```


4. di visual studio.code coding di buat

```
JS 01-intro-kompleksitas.js > ukurWaktu
1 function ukurWaktu(label, fn){
2   const mulai = Date.now();
3   fn
4   const selesai = Date.now();
5   console.log(`${label}: ${selesai - mulai} ms`);
6 }
7 const N =100_000;
8 function jumlahkanLinear(n) {
9   let total = 0;
10  for (let i = 1; i<= n; i++) total +=i;
11  return total;
12 }
13 function jumlahkanRumus(n) {
14  return (n * (n + 1)) / 2;
15 }
16 function cariPasangan(arr){
17   const pasangan = [];
18   for (let i = 0; i < arr.length; i++){
19     for (let j = i + 1; j < arr.length; i++){
20       if (arr[i] + arr [j] === 10) pasangan.push([arr[i],arr[j]]);
21     }
22   }
23   return pasangan;
24 }
25 const data = Array.from({length: 5000}, (_,i) => i);
26
27 console.log('===Perbandingan Waktu Eksekusi===');
28 ukurWaktu('0(1) jumlahkanRumus ', ()=> jumlahkanRumus(N));
29 ukurWaktu('0(n) jumlahkanLinear ', ()=> jumlahkanLinear(N));
30 ukurWaktu('0(n2) cariPasangan ', ()=> cariPasangan(data));
31
32 console.log('\nHasil sama?', jumlahkanLinear(100)===jumlahkanRumus(100));
```

output
terminal

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
C:\Program Files\nodejs\node.exe .\01-intro-kompleksitas.js
===Perbandingan Waktu Eksekusi===
0(1) jumlahkanRumus : 0 ms
0(n) jumlahkanLinear : 0 ms
0(n2) cariPasangan : 0 ms

Hasil sama? true
```

5. Buat file baru touch 02-destructuring-spread.js

```
JS 02-destructuring-spread.js > fibMemo
1 function ambilPertama(arr) { return arr[0]; }
2 function tambahkanItem(arr, item) {arr.push(item); }
3 function isGenap(n) {return n % 2 === 0; }
4
5 function binarySearch(arr, target){
6   let kiri = 0, kanan = arr.length -1;
7   let langkah = 0;
8   while (kiri <= kanan) {
9     langkah++;
10    const tengah = Math.floor((kiri + kanan) / 2);
11    if (arr[tengah] === target) {
12      console.log(' Ditemukan di indeks ${tengah} setelah ${langkah} langkah');
13      return tengah;
14    }
15    if (arr[tengah] < target) kiri = tengah + 1;
16    else kanan = tengah -1;
17  }
18  return -1;
19 }
20
21 function cariMax(arr) {
22   let maks = arr[0];
23   for (const x of arr) if (x > maks) maks = x;
24   return maks;
25 }
26
27 function bubbleSortNaif(arr) {
28   const a = [...arr];
29   for (let i = 0; i < a.length; i++)
30     for (let j = 0; j < a.length - i - 1; j++)
31       if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
32   return a;
33 }
34
35 function fibRekursif(n) {
36   if (n <= 1) return n;
37   return fibRekursif(n-1) + fibRekursif(n-2);
38 }
39
40 console.log('=== 0(1) - selalu cepat ===');
41 console.log(ambilPertama([10,20,30,40,50]));
42 console.log(isGenap(42));
43
44 console.log('\n=== O(log n) - Binary Search ===');
45 const sorted = Array.from({length:1_000_000}, (_,i)=>i);
46 binarySearch(sorted, 731_452);
47
48 console.log('\n=== O(n) - Linear Search ===');
49 console.log('Max dari 1000 elemen:');
50 cariMax(Array.from({length:1000}, ()=>Math.random()*1000|0));
51
52 console.log('\n=== O(n2) - Bubble Sort (kecil saja) ===');
53 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
54
55 console.log('\n=== O(2n) - Fibonacci Rekursif (n kecil!) ===');
56 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
57 console.log('');
58
59 console.log('\nWaktu fib(35) O(2n):');
60 let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');
61
62 console.log('Waktu fib(35) O(n) memoization:');
63 const memo = {};
```

6. Node 02 -destructuring.js di terminal

```
Hasil sama? true
PS D:\PRA_BASIS_DATA\2025573010038_Strukturdatalgoritma\praktikum-5> node 02-destructuring-spread.js
=== O(1) - selalu cepat ===
10
true
=== O(log n) - Binary Search ===
```

7. latihan-1-kompleksitas.js

```
JS latihan-1.js > ...
1  noconsole.log('\n latihan identifikasi kompleksitas-1.js');
2
3  function fungsiA(n){
4      return n * 2;
5  }
6
7  function fungsiB(n){
8      for(let i = 0; i < n; i++){
9          for(let j = 0; j < n; j++){
10             let x = i + j;
11         }
12     }
13 }
14
15 function fungsiC(n){
16     for(let i = 1; i < n; i *= 2){
17         let x = i;
18     }
19 }
31 function hitungKompleksitas(n, fn, bigO){
32     const start = Date.now();
33     fn(n);
34     const end = Date.now();
35
36     console.log("Big O:", bigO);
37     console.log("Waktu eksekusi:", (end - start), "ms");
38     console.log("-----");
39 }
40
41 let n = 50;
42
43 console.log("Fungsi A");
44 hitungKompleksitas(n, fungsiA, "O(1)");
45
46 console.log("Fungsi B");
47 hitungKompleksitas(n, fungsiB, "O(n^2)");
48
```

Output

```
PS D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\praktikum-5> node latihan-1.js

latihan identifikasi kompleksitas-1.js
Fungsi A
Big O: O(1)
Waktu eksekusi: 0 ms
-----
Fungsi B
Big O: O(n^2)
Waktu eksekusi: 0 ms
-----
Fungsi C
Big O: O(log n)
Waktu eksekusi: 0 ms
-----
Fungsi D
Big O: O(n^3)
Waktu eksekusi: 2 ms
```

7. touch 03-space-complexity.js

```
03-space-complexity.js > ...
1  function jumlahArray(arr) {
5  }
6  function duplikasiArray(arr) {
7      const baru = [];
8      for (const x of arr) baru.push(x * 2);
9      return baru;
10 }
11 function faktorialRekursif(n) {
12     if (n <= 1) return 1;
13     return n * faktorialRekursif(n - 1);
14 }
15
16 function faktorialIteratif(n) {
17     let hasil = 1;
18     for (let i = 2; i <= n; i++) hasil *= i;
19     return hasil;
20 }
21 const arr = [1,2,3,4,5,6,7,8,9,10];
22 console.log('Jumlah array:', jumlahArray(arr));
23 console.log('Duplikasi array:', duplikasiArray(arr));
24 console.log('Faktorial 10 rekursif:', faktorialRekursif(10));
25 console.log('Faktorial 10 iteratif:', faktorialIteratif(10));
26
41     return [arr[i], arr[j]];
42 }
43 }
44 }
45 return null;
46 }
47 function cariPasanganCepat(arr, target){
48     let seen = new Set();
49     for(let num of arr){
50         let pasangan = target - num;
51         if(seen.has(pasangan)){
52             return [pasangan, num];
53         }
54         seen.add(num);
55     }
56     return null;
57 }
58
59 let arr2 = [2,7,11,15];
60 let target = 9;
61
62 console.log("Hasil Lambat:" cariPasanganLambat(arr2, target));
```

```

75   let start2 = Date.now();
76   cariPasanganCepat(besar, targetBesar);
77   let end2 = Date.now();
78
79   console.log("Waktu Lambat:", (end1 - start1), "ms");
80   console.log("Waktu Cepat:", (end2 - start2), "ms");

```

Output

```

PS D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\praktikum-5> node 03-space-complexity.js
Jumlah array: 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif: 3628800
Faktorial 10 iteratif: 3628800
Elemen unik: 7

Cari pasangan angka
Hasil Lambat: [ 2, 7 ]
Hasil Cepat: [ 2, 7 ]
Waktu Lambat: 29 ms
Waktu Cepat: 2 ms

```

latihan .. bank account

```

82   console.log("\n=== Latihan bankaccount ===");
83   function cariPasanganLambat(arr, target) {
84     for (let i = 0; i < arr.length; i++) {
85       for (let j = i + 1; j < arr.length; j++) {
86         if (arr[i] + arr[j] === target) {
87           return [arr[i], arr[j]];
88         }
89       }
90     }
91     return null;
92   }
93   function cariPasanganCepat(arr, target) {
94     const seen = new Set();
95
96     for (const angka of arr) {
97       const pasangan = target - angka;
98
99       if (seen.has(pasangan)) {
100         return [pasangan, angka];
101       }
102       seen.add(angka);
103     }
104     return null;
105   }
106   const dataPasangan = [2, 7, 11, 15];
107   const targetPasangan = 9;
108   console.log("Hasil Lambat:", cariPasanganLambat(dataPasangan, targetPasangan));
109   console.log("Hasil Cepat :", cariPasanganCepat(dataPasangan, targetPasangan));
110

```

```

119 let selesai = Date.now();
120
121 console.log("\nHasil Lambat:", hasilLambat);
122 console.log("Waktu Lambat:", selesai - mulai, "ms");
123
124 mulai = Date.now();
125 const hasilCepat = cariPasanganCepat(dataBesar, targetTidakAda);
126 selesai = Date.now();
127
128 console.log("\nHasil Cepat:", hasilCepat);
129 console.log("Waktu Cepat:", selesai - mulai, "ms");
130
131 console.log("\nDiskusi:");
132 console.log("Fungsi cariPasanganCepat lebih baik karena hanya melakukan satu kali perulangan.");
133 console.log("Trade-off-nya adalah cariPasanganCepat membutuhkan memori tambahan karena menggunakan Set.");
134 console.log("Fungsi cariPasanganLambat lebih hemat memori, tetapi lebih lambat karena menggunakan nested loop.");

```

output terminal

```

=== Latihan bankaccount ===
Hasil Lambat: [ 2, 7 ]
Hasil Cepat : [ 2, 7 ]

Hasil Lambat: null
Waktu Lambat: 3 ms

Hasil Cepat: null
Waktu Cepat: 0 ms

Diskusi:
Fungsi cariPasanganCepat lebih baik karena hanya melakukan satu kali perulangan.
Trade-off-nya adalah cariPasanganCepat membutuhkan memori tambahan karena menggunakan Set.
Fungsi cariPasanganLambat lebih hemat memori, tetapi lebih lambat karena menggunakan nested loop.

```

Latihan1.js analisis dan refactor

<pre> tugas > JS tugas-1.js > intersectionLambat 1 console.log("=== Tugas 1: Analisis dan Refactor ==="); 2 // ===== 3 // FUNGSI A: INTERSECTION DUA ARRAY 4 // ===== 5 6 // Time Complexity: O(n²) 7 // Space Complexity: O(k), k = jumlah elemen yang sama 8 function intersectionLambat(arr1, arr2) { 9 const hasil = []; 10 11 for (const x of arr1) { 12 for (const y of arr2) { 13 if (x === y && !hasil.includes(x)) { 14 hasil.push(x); 15 } 16 } 17 } 18 19 return hasil; 20 } 21 // Time Complexity: O(n) </pre>	<pre> 31 } 32 33 return hasil; 34 } 35 // ===== 36 // FUNGSI B: KELOMPOK ANAGRAM 37 // ===== 38 39 // Time Complexity: O(n * k log k) 40 // n = jumlah kata, k = panjang rata-rata kata 41 // Space Complexity: O(n) 42 function kelompokAnagram(words) { 43 const map = new Map(); 44 45 for (const word of words) { 46 const key = word.split("").sort().join(""); 47 48 if (!map.has(key)) { </pre>
--	--

```

57 // =====
58 // FUNGSI C: ADA DUA ANGKA YANG JUMLAHNYA = KUADRAT ANGKA KETIGA
59 // =====
60
61 // Time Complexity:  $O(n^3)$ 
62 // Space Complexity:  $O(1)$ 
63 function cekKuadratLambat(arr) {
64   for (let k = 0; k < arr.length; k++) {
65     const target = arr[k] * arr[k];
66
67     for (let i = 0; i < arr.length; i++) {
68       for (let j = i + 1; j < arr.length; j++) {
69         if (arr[i] + arr[j] === target) {
70           return true;
71         }
72       }
73     }
74   }
75
76   return false;
77 }
78 // Time Complexity:  $O(n^2)$ 
79 // Space Complexity:  $O(n)$ 
80 function cekKuadratCepat(arr) {
81   for (let k = 0; k < arr.length; k++) {
82     const target = arr[k] * arr[k];
83     const seen = new Set();
84
85     for (const angka of arr) {
86       const pasangan = target - angka;

```

```

88     if (seen.has(pasangan)) {
89       return true;
90     }
91
92     seen.add(angka);
93   }
94 }
95
96   return false;
97 }
98
99 console.log("\nFungsi A:");
100 console.log(intersectionLambat([1, 2, 3, 4], [3, 4, 5, 6]));
101 console.log(intersectionCepat([1, 2, 3, 4], [3, 4, 5, 6]));
102
103 console.log("\nFungsi B:");
104 console.log(kelompokAnagram(["eat", "tea", "tan", "ate", "nat", "bat"]));
105
106 console.log("\nFungsi C:");
107 console.log(cekKuadratLambat([1, 2, 3, 4, 5]));
108 console.log(cekKuadratCepat([1, 2, 3, 4, 5]));
109
110 console.log("\n=== Benchmark ===");
111
112 const arrA1 = Array.from({ length: 3000 }, (_, i) => i);
113 const arrA2 = Array.from({ length: 3000 }, (_, i) => i + 1500);

```

```

115 let mulai = Date.now();
116 intersectionLambat(arrA1, arrA2);
117 console.log("Intersection Lambat:", Date.now() - mulai, "ms");
118
119 mulai = Date.now();
120 intersectionCepat(arrA1, arrA2);
121 console.log("Intersection Cepat :", Date.now() - mulai, "ms");
122
123 const kataBesar = ["eat", "tea", "tan", "ate", "nat", "bat"];
124
125 mulai = Date.now();
126 kelompokAnagram(kataBesar);
127 console.log("Kelompok Anagram:", Date.now() - mulai, "ms");
128
129 const arrC = Array.from({ length: 200 }, (_, i) => i + 1);
130
131 mulai = Date.now();
132 cekKuadratLambat(arrC);
133 console.log("Kuadrat Lambat:", Date.now() - mulai, "ms");
134
135 mulai = Date.now();
136 cekKuadratCepat(arrC);
137 console.log("Kuadrat Cepat :", Date.now() - mulai, "ms");

```

C:\Program Files\nodejs\node.exe .\tugas\tugas-1.js

=== Tugas 1: Analisis dan Refactor ===

Fungsi A:

```

> (2) [3, 4]
> (2) [3, 4]

```

Fungsi B:

```

> (3) [Array(3), Array(2), Array(1)]

```

Fungsi C:

```

2 true

```

=== Benchmark ===

Intersection Lambat: 39 ms

Intersection Cepat : 2 ms

Kelompok Anagram: 0 ms

Kuadrat Lambat: 1 ms

Kuadrat Cepat : 0 ms

Latihan 2. Visualisasi pertumbuhan big o

```
tugas > js tugas-2.js > fn_OnLogn
1  console.log("=== Tugas 2: Visualisasi Pertumbuhan Big O ===");
2  function fn_O1(n) {
3    return n + 1;
4  }
5  function fn_On(n) {
6    let total = 0;
7
8    for (let i = 0; i < n; i++) {
9      total += i;
10   }
11   return total;
12 }
13 function fn_OnLogn(n) {
14   let total = 0;
15   const logN = Math.floor(Math.log2(n));
16
17   for (let i = 0; i < n; i++) {
18     for (let j = 0; j < logN; j++) {
19       total++;
20     }
21   }
22   return total;
23 }
24 function fn_On2(n) {
25   let total = 0;
26
27   for (let i = 0; i < n; i++) {
28     for (let j = 0; j < n; j++) {
29       total++;
30     }
31   }
32   return total;
33 }
34 function ukurWaktu(nama, fn, n) {
35   const mulai = Date.now();
36   fn(n);
37   const selesai = Date.now();
38   console.log(`${nama} untuk n=${n}: ${selesai - mulai} ms`);
39 }
40 function benchmarkSemua(ukuranData) {
41   for (const n of ukuranData) {
42     console.log(`\nUkuran data: ${n}`);
43
44     ukurWaktu("O(1)", fn_O1, n);
45     ukurWaktu("O(n)", fn_On, n);
46     ukurWaktu("O(n log n)", fn_OnLogn, n);
47     ukurWaktu("O(n^2)", fn_On2, n);
48   }
49 }
50
51 benchmarkSemua([100, 500, 1000, 5000, 10000]);
52 /*
53 Komentar hasil pengamatan:
54
55 1. O(1) cenderung selalu sangat cepat karena hanya menjalankan satu operasi.
56 2. O(n) bertambah seiring bertambahnya ukuran input.
57 3. O(n log n) lebih lambat dari O(n), tetapi masih lebih baik daripada O(n^2).
58 4. O(n^2) paling lambat karena menggunakan nested loop n x n.
59 5. Pola pertumbuhan ini konsisten dengan teori Big O.
60 */
```


output terminal

```
C:\Program Files\nodejs\node.exe .\tugas\tugas-2.js
=== Tugas 2: Visualisasi Pertumbuhan Big O ===

Ukuran data: 100
O(1) untuk n=100: 0 ms
O(n) untuk n=100: 0 ms
O(n log n) untuk n=100: 1 ms
O(n2) untuk n=100: 1 ms

Ukuran data: 500
O(1) untuk n=500: 0 ms
O(n) untuk n=500: 0 ms
O(n log n) untuk n=500: 0 ms
O(n2) untuk n=500: 2 ms

Ukuran data: 1000
O(1) untuk n=1000: 0 ms
O(n) untuk n=1000: 0 ms
O(n log n) untuk n=1000: 0 ms
O(n2) untuk n=1000: 3 ms

Ukuran data: 5000
O(1) untuk n=5000: 0 ms
O(n) untuk n=5000: 0 ms
O(n log n) untuk n=5000: 0 ms
O(n2) untuk n=5000: 45 ms

Ukuran data: 10000
O(1) untuk n=10000: 0 ms
O(n) untuk n=10000: 0 ms
O(n log n) untuk n=10000: 1 ms
O(n2) untuk n=10000: 165 ms
```

Cara push ke git hub

1. Ketik cd nama foldercd ..

git status

Langkah 2 Tambahkan folder praktikum-5/

git add praktikum-5/

Langkah 3 Commit dan push.

git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"

git push origin main

```
HP@LAPTOP-5IBPM001 MINGW64 ~
$ cd /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma

HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ npm init -y
Wrote to D:\PRA_BASIS_DATA\2025573010038_Strukturdataalgoritma\package.json:

{
  "name": "2025573010038_strukturdataalgoritma",
  "version": "1.0.0",
  "description": "struktur data logika dan algortma",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/nuraidasafitri193-sys/2025573010038_Strukturdataalgoritma.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/nuraidasafitri193-sys/2025573010038_Strukturdataalgoritma/issues"
  },
  "homepage": "https://github.com/nuraidasafitri193-sys/2025573010038_Strukturdataalgoritma#readme"
}

HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it

HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ git commit -m "praktikum 5: analisis kompleksitas algoritma"
[main eb8c898] praktikum 5: analisis kompleksitas algoritma
10 files changed, 617 insertions(+)
create mode 100644 praktikum-5/01-intro-kompleksitas.js
create mode 100644 praktikum-5/02-destructuring-spread.js
create mode 100644 praktikum-5/03-space-complexity.js
create mode 100644 praktikum-5/04-refactor.js
create mode 100644 praktikum-5/latihan-1.js
create mode 100644 praktikum-5/package.json
create mode 100644 praktikum-5/spread.js
create mode 100644 praktikum-5/tugas/tugas-1.js
create mode 100644 praktikum-5/tugas/tugas-2.js
create mode 100644 praktikum-5/tugas1.js

HP@LAPTOP-5IBPM001 MINGW64 /d/PRA_BASIS_DATA/2025573010038_Strukturdataalgoritma
(main)
$ git push origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 12 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (14/14), 5.81 KiB | 1.16 MiB/s, done.
Total 14 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/nuraidasafitri193-sys/2025573010038_Strukturdataalgoritma.
git
5d1ed28..eb8c898 main -> main
```

BAB IV KESIMPULAN

Dari praktikum ini dapat disimpulkan bahwa:

- Kompleksitas algoritma sangat mempengaruhi performa program
- Algoritma dengan kompleksitas kecil lebih efisien untuk data besar
- Hasil pengujian sesuai dengan teori Big O
- $O(n^2)$ adalah yang paling tidak efisien dibandingkan yang lain

Analisis Data

Berdasarkan hasil pengujian, terlihat bahwa setiap algoritma memiliki performa yang berbeda tergantung kompleksitasnya.

- **Fungsi A (Intersection):**
Versi $O(n^2)$ lebih lambat karena nested loop, sedangkan $O(n)$ lebih cepat karena menggunakan Set.
- **Fungsi B (Anagram):**
Cukup efisien, tetapi waktu bertambah jika jumlah dan panjang kata meningkat karena proses sorting.
- **Fungsi C (Kuadrat):**
 $O(n^3)$ sangat lambat, sedangkan $O(n^2)$ lebih cepat karena mengurangi jumlah loop.
- **Benchmark Big O:**
 $O(1)$ paling cepat dan stabil, $O(n)$ meningkat linear, $O(n \log n)$ sedang, dan $O(n^2)$ paling lambat.